Setting Up Telegram Integration with Self-Hosted n8n (Docker)

This documents how I got the Telegram Trigger node working on my self-hosted n8n instance running in Docker on Ubuntu.

Overview

Telegram delivers messages to bots via webhooks, which means Telegram's servers need to be able to reach n8n over the public internet with a valid HTTPS URL. Since n8n runs locally in Docker, I used **ngrok** to create a secure tunnel from a public URL down to my local port.

Architecture:

```
Telegram → https://<ngrok-domain>.ngrok-free.dev → ngrok tunnel →
localhost:5678 → n8n container
```

Prerequisites

- Docker installed and running
- A Telegram account
- An ngrok account (free tier works)

Step 1: Create the Telegram Bot

1. Open Telegram and search for **@BotFather**.
2. Send `/newbot` and follow the prompts:
    - Choose a display name
    - Choose a username ending in `bot` (e.g. `MyN8nBot`)
3. BotFather replies with an **API token** — save this, it's needed for the n8n credential.

Step 2: Install and Configure ngrok

Install via snap:

```
sudo snap install ngrok
```

Sign up for a free account at https://dashboard.ngrok.com/signup, then get the auth token:

```
ngrok config add-authtoken <your-token-here>
```

Important: ngrok's free tier now requires claiming a static domain before tunneling.

1. Go to https://dashboard.ngrok.com/domains
2. Click **Create Domain** to get a free static domain (e.g. `stylist-juggle-oversight.ngrok-free.dev`)

3. This domain won't change between restarts, unlike the old random-URL free tier.

Step 3: Run n8n in Docker with the Correct Environment Variables

n8n needs to know its own public-facing URL so it can correctly register webhooks with Telegram.

```
docker run -d \
  --name n8n \
  -p 5678:5678 \
  -e WEBHOOK_URL=https://<your-ngrok-domain>/ \
  -e N8N_HOST=<your-ngrok-domain> \
  -e N8N_PROTOCOL=https \
  -v n8n_data:/home/node/.n8n \
  docker.n8n.io/n8nio/n8n
```

Notes:

- `-d` runs the container in detached (background) mode so it survives closing the terminal.
- `-v n8n_data:/home/node/.n8n` mounts a named volume so workflows and credentials persist across container recreation.
- If a container named `n8n` already exists, remove it first: `docker stop n8n && docker rm n8n`.

Verify it started cleanly:

```
docker logs -f n8n
```

Look for a line like:

```
Editor is now accessible via:
https://<your-ngrok-domain>
```

Step 4: Start the ngrok Tunnel

In a separate terminal (must stay open/running):

```
ngrok http --domain=<your-ngrok-domain> 5678
```

Confirm the `Forwarding` line shows your domain pointing to `localhost:5678`.

This terminal (or an equivalent background process) needs to stay alive — if the tunnel drops, Telegram can no longer reach n8n even if the container is running fine.

Step 5: Configure Telegram in n8n

1. Open the n8n editor at `https://<your-ngrok-domain>`.
2. Go to **Credentials** → **Add Credential** → **Telegram**, paste the bot token from Step 1, and save.
3. Create or open a workflow.

4. Add a **Telegram Trigger** node, select the credential, and set **Trigger On** to `Message`.
5. **Activate** the workflow (toggle top-right). This registers the webhook URL with Telegram automatically.

Step 6: Test

1. Open Telegram, find the bot by username, and send it a message.
2. In n8n, check the **Executions** tab — a new execution should appear containing the message data.

Troubleshooting Reference

Symptom	Cause	Fix
`ERR_NGROK_3200` (endpoint offline)	ngrok tunnel process was closed/killed	Restart with `ngrok http --domain=<domain> 5678`
`ERR_NGROK_15013` on first run	Free tier requires a claimed static domain	Create one at https://dashboard.ngrok.com/domains and use `--domain=` flag
No execution appears after sending a message	Workflow not activated, or `WEBHOOK_URL`/ngrok domain mismatch	Confirm workflow is Active; verify webhook via `https://api.telegram.org/bot<TOKEN>/getWebhookInfo`
Container name conflict on `docker run`	An old container named `n8n` already exists	`docker stop n8n && docker rm n8n`, then re-run

Notes for Longer-Term Use

- Free ngrok static domains persist across restarts, so `WEBHOOK_URL` doesn't need to change once set — but the ngrok process itself still needs to be kept running (e.g. via `nohup` or a systemd service) to stay online.
- For a more permanent/production setup, consider switching to **Cloudflare Tunnel** with an owned domain, which doesn't depend on a client process tied to a single terminal session in the same way.
- `WEBHOOK_URL` is deprecated in newer n8n versions in favor of `N8N_WEBHOOK_URL` — worth migrating when convenient.